BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Bubble Sort

Name _____ **Class** _____ **Date** _____ **Time** _____ **Teacher** _____

Do Now - Four retrieval questions

BLACK PEN: answer four questions before checking the answers.

1. What must be true before binary search is used?

2. How can a subprogram make code easier to maintain?

A) Copy logic into every branch
B) Hide all useful names
C) Remove all testing
D) Change shared logic once

3. Why is indentation useful?

4. An unusual validation rule needs updating next year. "A comment changes the next variable's value." Correct this. How could a comment help?

Today's objectives

Explain comparisons of neighbouring elements. Trace swaps during each pass.

Key vocabulary - Read, cover, recall

Bubble sort
Repeatedly compares and swaps adjacent values to order data.

Adjacent
Next to each other in the array.

Swap
Exchange positions of two values.

Pass
One traversal through consecutive pairs.

Write one key term and its meaning in your own words:

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Learn and practise

New learning - Key ideas

Look at adjacent pairs	Bubble sort checks neighbouring elements from left to right, comparing a[i] with a[i+1].
Swap only if out of order	For ascending order, swap a pair when the left value is greater than the right value.
Complete one pass	Continue through pairs. Larger values move towards the rightmost end after a full pass.
Repeat to finish sorting	Repeat passes as necessary until all values are in order; an optimised version can stop when a pass makes no swaps.

I Do - One worked example

Trace the first full ascending bubble-sort pass on [5,2,4]. State each swap and the resulting array. [4 marks]

1. Compare 5 and 2: 5 > 2 [1].

2. Swap to [2,5,4] [1].

3. Compare 5 and 4: 5 > 4 [1].

4. Swap to [2,4,5]; the pass is complete [1].

We Do - Guided questions

BLACK PEN: use the model, then explain your answer.

1. Trace the first ascending bubble-sort pass on [6,1,3]. Give the array after each swap. [4 marks]

2. For [2,4,3], explain whether the first pair swaps and what the pass produces. [3 marks]

3. Explain why an array may need more than one bubble-sort pass. [2 marks]

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Think and write

Code First - Predict > Write > Test > Improve

YOUR TASK: Complete the missing comparison for one ascending bubble-sort pass.

```
array sizes = [6,1,3]
for i = 0 to 1
if ... then
temp = sizes[i]
sizes[i] = sizes[i+1]
sizes[i+1] = temp
endif
next i
print(sizes[0])
print(sizes[1])
print(sizes[2])
```

BLACK PEN: predict or complete the code; test, then improve.

For extended answers, continue on lined paper if needed.

You Do - Independent exam questions

Q2. State when two adjacent numbers swap during an ascending bubble sort. [2 marks]

Q3. Complete one bubble-sort pass on [7,2,5]. State the two intermediate arrays. [4 marks]

Q4. Explain how bubble sort orders [4,3,1], showing the results after each necessary swap and pass. [6 marks]

Exit Ticket - True or False?

Circle T or F before reviewing Page 4.

1. Bubble sort compares neighbouring elements and may need several passes. ☐ T ☐ F

2. Bubble sort checks neighbouring elements from left to right, comparing a[i] with a[i+1]. ☐ T ☐ F

3. For ascending order, swap a pair when the left value is greater than the right value. ☐ T ☐ F

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & improve

GREEN PEN: tick correct marking points, then correct errors.

Choose your hardest question to rewrite in full below.

Do Now - Answers

Q1 The list must be sorted. This lets the search safely rule out half the remaining items. ☐

Q2 D - Change shared logic once ☐

Q3 Indentation shows blocks and nesting clearly. Some languages require it. ☐

Q4 A comment explains why code works or what it is for. Clear comments help others understand and change it. ☐

Code First - Model

```
array sizes = [6,1,3]
for i = 0 to 1
if sizes[i] > sizes[i+1] then
temp = sizes[i]
sizes[i] = sizes[i+1]
sizes[i+1] = temp
endif
next i
print(sizes[0])
print(sizes[1])
print(sizes[2])
```

Output: 1
3
6

We Do - Check the model

1. 6 > 1 so swap [1,6,3] [2]. Then 6 > 3 so swap [1,3,6] [2]. ☐

2. 2 < 4, so no swap [1]. Compare 4 and 3 and swap [1], producing [2,3,4] [1]. ☐

3. One pass moves larger elements rightwards [1] but smaller values can still be out of order and need further passes [1]. ☐

Exit Ticket - Check

1. TRUE 2. TRUE 3. TRUE

You Do - Self-mark

Q2 The left value is greater than the right value [1], so their positions are exchanged [1]. ☐

Q3 7 > 2 [1], swap [2,7,5] [1]; 7 > 5 [1], swap [2,5,7] [1]. ☐

Q4 Compare 4 and 3 [1], swap [3,4,1] [1]. Compare 4 and 1 [1], swap [3,1,4] [1]. Next pass compare 3 and 1 [1], swap [1,3,4], now sorted [1]. ☐

Try the question again

1. Copy the FULL question you found most difficult:

2. In GREEN PEN, write a complete new answer: